

Indexing

1. What Indexes Are For

An index is an **auxiliary access structure** — a separate, additional file that speeds up retrieval on a specific field, **without changing** the physical order of the actual data file.

The core goal (recall from the File Organization topic): minimize the number of block accesses needed to find a record. Indexes achieve this by being much **smaller** than the data file itself (they store only the search key + a pointer, not the whole record), so binary-searching the index is far cheaper than binary-searching (or worse, linearly searching) the actual data.

2. Single-Level Ordered Indexing

Primary Index

Built on the **ordering key** of a file that's physically sorted by that same field.

- **Sparse index** — one index entry **per block** of the data file (pointing to the first/"anchor" record of that block), not per record.
- Because the data file is already sorted by this field, a sparse index is sufficient: once you find the right block via the index, a short scan within that one block finds the exact record.

Search using a primary index: binary search the (much smaller) index file, then read the **one** data block it points to.

Problems: inserting a record in its correct sorted position still requires shifting records (same issue as ordered files in general) — mitigated the same way, via an unordered overflow file, or by using deletion markers instead of physical deletion.

Clustering Index

Built on a **non-key** ordering field (the data file is sorted by this field, but the field is **not unique** — many records can share a value).

- Also sparse — but here, one entry **per distinct value** of the clustering field (not per block), since multiple blocks might hold the same value's records.

Dept 10 → Block 1 (all Dept 10 employees start here)
Dept 20 → Block 3
Dept 30 → Block 5

Comparison: Primary vs. Clustering Index

Feature	Primary Index	Clustering Index
Ordering field	Unique (a key)	Non-unique
Entry per	Data block	Distinct value of the clustering field
Density	Sparse	Sparse
Retrieves	A single record	Potentially many records sharing that value

Problem: inserting a new record into the middle of a clustering-field group can require restructuring — a common fix is reserving a separate block (or free space) for each distinct clustering-field value, so new records for that value have somewhere to go without disturbing neighbors.

Secondary Index

An **additional**, alternative access path on a field that is **neither** the primary ordering field nor the clustering field — the data file is **not** sorted by this field at all.

- Must be **dense** — one entry **per record** (not per block/value) — because, unlike primary/clustering indexes, there's no physical ordering to exploit; every single record's location must be recorded explicitly, or it simply couldn't be found at all via this index.
- You can build **many** secondary indexes on the same file (one per field you commonly search on) — like having several different indexes in the back of a book, each organized by a different criterion.
- For a **non-key** secondary index field (multiple records can share a value), a common efficient technique: the index entry points to a single **bucket** of pointers to all matching records, rather than making the index itself variable-length.

Comparison: Primary vs. Secondary Index

Feature	Primary Index	Secondary Index
Requires data file ordered by index field?	Yes	No
Density	Sparse	Dense
Search speed	Faster (fewer index entries, plus data file itself aids the final scan)	Slightly slower (larger, dense index) but far more flexible

Interview soundbite: "The type of index — primary, clustering, or secondary — isn't really a design choice made in isolation; it's determined entirely by the *relationship* between the index field and the file's physical ordering. Sparse-vs-dense follows directly from that: you can only get away with sparse when the file's own physical order does part of the work for you."

3. Multilevel Indexes

The problem

A single-level index, once large enough, still requires $\log_2(b)$ block accesses via binary search — for a genuinely large file, this "small" index can itself span hundreds of blocks.

The idea

Build an index on the index — recursively — forming a hierarchy, where the top level is small enough to fit in a **single block**.

```
Level t   (top level — fits in ONE block, search always starts here)
|
Level t-1
|
...
|
Level 1   (a sparse index directly over the data file)
|
Data file
```

Search procedure: read the top-level block → it points to the right block one level down → read that → continue descending one block per level → finally read the **one** data block containing the target record.

Total block accesses = t (index levels) + 1 (data block)

Fan-out

Fan-out (fo) — the number of index entries that fit in one block; this determines how quickly each level narrows the search.

Worked comparison (from a dense secondary index spanning 442 blocks):

	Single-level (binary search)	Multilevel
Search cost	$\log_2(442) \approx 9$ index accesses + 1 data = 10 total	≈ 3 index accesses + 1 data = 4 total

A multilevel index turns the search cost from $\log_2(b)$ into $\log_{f_o}(b)$ — since f_o is typically much larger than 2 (a block can usually hold many more than 2 index entries), this is a dramatic improvement, cutting access counts by more than half in this example.

ISAM (Indexed Sequential Access Method) — a classic real-world implementation: an ordered data file combined with a multilevel primary index (IBM's ISAM famously used cylinder/track-based index levels matching physical disk geometry).

4. The Problem With *Static* Multilevel Indexes — and the Fix

A **fixed** multilevel index (built once, for a given data size) becomes inefficient as data is inserted/deleted — it may require **complete reorganization** to stay balanced and efficient, similar to how a sorted file needs reorganization.

The fix: a *dynamic* multilevel index that automatically rebalances itself as data changes — this is exactly what **B-Trees** and **B+-Trees** are.

Search trees — the underlying idea

A node holds sorted keys $K_1 < K_2 < \dots < K_{q-1}$ and q pointers $P_1 \dots P_q$, where each P_i leads to a subtree containing only values in the appropriate range relative to the surrounding keys.

Problem with a *basic (unbalanced)* search tree: repeated insertions/deletions can make some paths deeper than others, degrading worst-case search time, and can leave nodes mostly empty, wasting space — hence the need for a *balanced* variant.

B-Trees

A **balanced** search tree, formally guaranteeing:

1. Keys within a node are sorted.
2. **Each internal node has at most p pointers** (this p is the tree's **order**).
3. **Every non-root, non-leaf node has at least $\lceil p/2 \rceil$ pointers** — guarantees every node stays at least half-full, bounding wasted space.
4. **The root has at least 2 pointers** (unless it's the only node in the tree).
5. **A node with q pointers has exactly $q-1$ keys.**
6. **All leaf nodes are at the same level** — this is precisely what *balanced* means here, and it guarantees worst-case search time is bounded, not just average-case.

Each node stores both **tree pointers** and **data pointers** (pointers directly to actual records/data blocks) intermixed at every level, including internal nodes.

B-Tree's limitation: since data pointers exist at *every* level (not just leaves), a **range query** (e.g., "all salaries between X and Y") can't simply walk along one level efficiently — you'd need to traverse

up and down across different levels to collect all qualifying records in order.

B+-Trees — the practical default in real DBMSs

Fixes the range-query weakness by **keeping actual data pointers only in leaf nodes**, and **linking the leaf nodes together** in a chain.

- **Internal nodes** — pure navigation: sorted keys + child pointers only, **no data pointers at all**. Search rule: for a search value x , if $K_{i-1} < x \leq K_i$, follow pointer P_i .
- **Leaf nodes** — hold the actual data pointers **and** a pointer to the **next leaf node** in sorted order.
- Same balance/fill-factor rules as a B-tree (max p pointers, minimum $\lceil p/2 \rceil$, etc.) apply to internal nodes.

Why this fixes range queries: once you've descended to the correct starting leaf, you can simply **follow the leaf-to-leaf chain** sequentially to collect every subsequent qualifying record, without ever needing to go back up the tree — a range scan becomes a simple linked-list walk at the leaf level.

Interview soundbite: "The single sentence that captures the entire B-tree → B+-tree evolution: B-trees put data pointers everywhere, which wastes internal-node space and makes range scans awkward; B+-trees push all data pointers down to the leaves and link the leaves together, which packs more keys per internal node (better fan-out, shallower tree) *and* turns range queries into a cheap sequential leaf scan. This is exactly why virtually every real-world relational database index (and many NoSQL ones) is a B+-tree, not a plain B-tree."

5. Indexes on Multiple Keys (Composite Indexes)

- **Multiple single-field indexes** — separate indexes on, say, `Dno` and `Age` independently.
- **Composite (multikey) index** — **one** index combining both attributes together, e.g., an index on `<Dno, Age>`.

Why composite indexes help: if queries commonly filter on **combinations** of fields together (e.g., `Dno=4 AND Age=59`), a composite index directly narrows to exactly that combination — far more selective than intersecting the results of two separate single-field indexes.

Key limitation: a composite index on `<A1, A2, ..., An>` is only efficient when the query uses the **leftmost** attribute(s) of the index, in order. An index on `<Dno, Age>` helps `Dno=4` alone, and `Dno=4 AND Age=59` together — but **does not help** a query filtering on `Age` alone (Age isn't the leading attribute, so its values are scattered arbitrarily throughout the index, not grouped together).

Partitioned Hashing

Extends hashing to composite keys: generate a **separate hash for each attribute**, then **concatenate** them into one combined bucket address.

Worked example: key `<Dno, Age>`, with `Dno` hashed to 3 bits and `Age` hashed to 5 bits.

```
Dno=4   → 100
Age=59  → 10101
Combined bucket address = 100 || 10101 = 10010101
```

- ✓ Extends cleanly to many attributes; retrieves exact combinations quickly; no need for separate per-attribute indexes.
- ✗ **No range queries at all** — only exact-match (equality) conditions work, since hashing destroys any notion of ordering (you can search `Dno=4 AND Age=59`, but never `Age > 40`).

Grid Files

A **multi-dimensional** structure — each attribute gets its own **linear scale (axis)**, divided into intervals; each cell in the resulting grid corresponds to one storage bucket.

Worked example: X-axis = `Dno`, Y-axis = `Age`. A record `(Dno=4, Age=59)` maps to a specific grid cell (e.g., cell `(1,5)`), which points to a specific bucket.

- Well-suited for **multi-key** access where you want reasonably efficient retrieval on *any* combination of the indexed attributes (unlike composite indexes' strict leftmost-prefix limitation).

Summary Comparison

Index type	Data file ordering required?	Density	Handles range queries?
Primary	Yes (by key)	Sparse	Yes
Clustering	Yes (by non-key field)	Sparse	Yes
Secondary	No	Dense	Yes
Multilevel (B-tree/B+-tree)	No	N/A (tree structure)	B-tree: awkwardly; B+-tree: efficiently
Partitioned hashing	No	N/A (hash buckets)	No — equality only
Grid file	No	N/A (multi-dim grid)	Reasonably, on any indexed combination

Interview Questions With Answers

Q1. Why must a secondary index always be dense, while a primary index can be sparse?

Answer: A primary index is built on a field the data file is physically sorted by, so once the index points to the correct block, a short local scan within that block finds the exact record — the file's own order does the "fine-grained" part of the work, so the index only needs one entry per block (sparse). A secondary index is built on a field the data file has no physical relationship to at all — records with any given value of that field could be scattered anywhere across the file — so there's no physical ordering to exploit, meaning every single record's location must be explicitly recorded in the index, or it simply couldn't be found through that index at all.

Q2. What's the key structural difference between a primary index and a clustering index, given both are sparse?

Answer: A primary index is built on a unique ordering key — one index entry corresponds to one data block, and within that block there's exactly one matching record. A clustering index is built on a non-unique ordering field — one index entry corresponds to one distinct *value* of that field (which may span multiple data blocks, since many records can share that value), and a lookup can return multiple matching records rather than just one.

Q3. Why does fan-out matter so much for multilevel index performance, and how does it change the complexity of a search?

Answer: Fan-out is the number of index entries that fit in a single block, which directly determines how much a search space shrinks with each level descended. A single-level index requires $\log_2(b)$ block accesses via binary search; a multilevel index instead requires $\log_{fo}(b)$ accesses, descending one block per level. Since fan-out is typically much larger than 2 (a block can usually hold many index entries, not just two), $\log_{fo}(b)$ is dramatically smaller than $\log_2(b)$ for the same file size — this is exactly why a multilevel index can turn a 9-10 block-access search into a 3-4 block-access search for the same underlying data.

Q4. Why does a fixed (static) multilevel index degrade over time, and what specific property of B-trees fixes this?

Answer: A static multilevel index is built once for a given data size; as records are inserted and deleted, the index can become unbalanced or require complete reorganization to stay efficient, since it has no built-in mechanism to adapt its own structure incrementally. B-trees fix this by enforcing balance as a formal invariant — every leaf node is guaranteed to be at the same level, and every node (except the root) is guaranteed to stay at least half-full — and by handling insertions/deletions through *local* node splits/merges rather than requiring the whole structure to be rebuilt, so the tree stays efficient and balanced continuously, without ever needing a full reorganization pass.

Q5. Explain precisely why B+-trees handle range queries more efficiently than plain B-trees.

Answer: In a plain B-tree, data pointers exist at every level (internal nodes as well as leaves), so records satisfying a range condition can be scattered across multiple levels of the tree — collecting them in order would require awkwardly traversing up and down between levels. In a B+-tree, all data pointers are pushed down to exist only in leaf nodes, and the leaf nodes are explicitly linked together in sorted order via a chain of pointers. This means that once a range query locates its starting point (via a normal tree descent), it can simply follow the leaf-to-leaf chain sequentially to collect every subsequent qualifying record, without ever needing to revisit higher tree levels — turning a range scan into a cheap, purely sequential leaf-level walk.

Q6. Why do internal nodes in a B+-tree typically allow a higher fan-out than equivalent B-tree nodes of the same size?

Answer: B-tree internal nodes must store both key values *and* data pointers (to actual records) alongside their child (tree) pointers, consuming space per entry. B+-tree internal nodes store *only* keys and child pointers — no data pointers at all, since those are pushed exclusively to the leaves — so, for the same fixed block size, a B+-tree internal node can fit more keys/pointers than a B-tree node would, giving it a higher fan-out. A higher fan-out directly means a shallower tree for the same amount of data, which means fewer block accesses per search.

Q7. Why is a composite index on `<Dno, Age>` useless for a query that filters only on Age ?

Answer: A composite index physically orders its entries first by the leading attribute (Dno), and only orders by the second attribute (Age) *within* each group of matching Dno values. Entries with the same Age value but different Dno values end up scattered throughout the index, not grouped together at all — so there's no way to efficiently locate "all records with Age=59" without effectively scanning the entire index, since Age values aren't contiguous or ordered independently of Dno. This is exactly the "leftmost prefix" limitation: a composite index is only useful for queries that constrain its attributes starting from the leftmost one, in order.

Q8. Why can't partitioned hashing support range queries, even though it's specifically designed for composite (multi-attribute) keys?

Answer: Partitioned hashing computes a bucket address by hashing each attribute independently and concatenating the results — and hash functions, by design, deliberately scramble input values so that even numerically close inputs (e.g., Age=40 and Age=41) can map to completely unrelated hash outputs and therefore unrelated bucket addresses. Since there's no preserved relationship between an attribute's actual value and its position in the address space, there's no way to identify "all buckets containing Age > 40" without checking every possible bucket — range conditions require some notion of ordering being preserved by the structure, which hashing explicitly does not provide; only exact-match equality conditions (which just need to compute the one specific hash and go directly there) work.

Q9. Scenario: A table has queries that sometimes filter on `Department` alone, sometimes on `Department AND Age` together, and sometimes on `Age` alone. Using only composite indexes, can one single index efficiently serve all three query patterns? What would you actually do?

Answer: A single composite index cannot serve all three patterns efficiently, due to the leftmost-prefix limitation. An index on `<Department, Age>` would efficiently serve "Department alone" and "Department AND Age together," but would be nearly useless for "Age alone," since Age values aren't grouped independently of Department within that index. Reversing it to `<Age, Department>` would flip the problem — now "Age alone" and "Age AND Department together" work well, but "Department alone" wouldn't. The practical solution is to build **two separate indexes**: a composite index on `<Department, Age>` (covering the first two query patterns) and a separate single-field index on `Age` alone (covering the third) — accepting the extra storage and maintenance overhead of two indexes as the cost of efficiently serving all three distinct query patterns, since no single composite ordering can satisfy leftmost-prefix requirements for both attribute orderings simultaneously.